



SPRING-BOOT 3.5 → 4.0 · MINI GUIDE



Observability & Ops: 3.5 → 4.0

Six changes your dashboards will notice first

One cut from the full Spring Break Cheat Sheets. Every card verified against Spring Boot 3.5.16 and 4.0.7. Real error output, real fixes, from the spring-break test suite. Steve Poole.

About this mini guide

This is one cut from **Spring Break Cheat Sheets**, the field guide to every breaking change between Spring Boot 3.5 and 4.0. Each card covers one change, verified by running real code on 3.5.16 and 4.0.7. One change, one card, one page: the error you'll see, the fix, and the traps.

HOW TO READ A CARD

TIER	How it fails. Tier 1 won't build, Tier 2 won't start, Tier 3 behaves differently. Tier 3 is the dangerous one: nothing goes red, your data just changes shape.
IMPACT	Effort per occurrence once found. S is find-and-replace, M needs an hour of care, L needs a plan.
OPENREWRITE	The badge means the Boot 4.0 recipe fixes this one for you. Run it before touching anything by hand.
WHAT YOU'LL SEE	The real output, so you can match it against your own build.
THE FIX	Before and after code, plus the steps when a rename isn't enough.
WATCH OUT	The traps that make this change look like a different bug.
VERIFY	The command that proves you're done.

THE SERIES

Spring Batch: 3.5 → 4.0	Eight changes between your jobs and 4.0
Hibernate & JPA: 3.5 → 4.0	Ten data-layer changes, mostly the quiet kind
Security & SSO: 3.5 → 4.0	Seven auth changes you cannot ignore
Data & Messaging: 3.5 → 4.0	Nine changes across your stores and streams
Testing: 3.5 → 4.0	Seven test-suite breaks, one clean build away
Observability & Ops: 3.5 → 4.0	Six changes your dashboards will notice first THIS GUIDE
Jackson & JSON: 3.5 → 4.0	Nine JSON changes everyone hits
Web & HTTP: 3.5 → 4.0	Eight changes at the HTTP layer
Core Platform & Build: 3.5 → 4.0	Twenty platform changes underneath everything
The Big Jobs: L-Impact Changes	The six changes that eat your sprint

The full guide collects every change in one book, with a complete planning matrix and the OpenRewrite robot list. github.com/spoole167/spring-break



About HeroDevs

HeroDevs keeps critical open source software secure after the community stops patching it. Never-Ending Support (NES) covers CVEs in Spring Boot 3.5, Java, and dozens of other end-of-life frameworks, so your team migrates on its own schedule instead of the vendor's deadline.

This book exists because migrations fail from surprise more often than difficulty. Every card is a fact, not a guess: verified by running real code on Spring Boot 3.5.16 and 4.0.7.

The complete book, every subsystem in one volume, is coming to LeanPub. Watch github.com/spoole167/spring-break for the link.

herodevs.com

The Planning Matrix

Every change in this book, in book order. Impact is the effort per occurrence once found (S small, M moderate, L large), not how many you have. Auto means the OpenRewrite Boot 4.0 recipe handles it.

6 CHANGES

4 × TIER 2 - WON'T START

2 × TIER 3 - BEHAVES DIFFERENTLY

TIER	CHANGE	IMPACT	AUTO
2	Actuator Endpoint @Nullable: org.springframework.lang Replaced by JSpecify	S	•
2	AspectJ Weaving Required for @Observed	S	•
2	Controller & View Spans Disabled by Default	S	•
2	OTLP/HTTP Now Primary Export Protocol	S	•
3	Liveness and Readiness Probes Enabled by Default	S	•
3	Controller/View Spans Silently Disabled	M	•

The Robot List vs The Human List

The left column is what the OpenRewrite Boot 4.0 recipe fixes while you get coffee. The right column needs a person who understands your code. This split is the honest size of the migration: the renames are free, the behaviour changes are the work.

THE ROBOT LIST

Run the recipe, review the diff, done.

THE HUMAN LIST

Read the card. Check your code. Decide.

Actuator Endpoint @Nullable: org.springframework.lang
Replaced by JSpecify

AspectJ Weaving Required for @Observed

Controller & View Spans Disabled by Default

OTLP/HTTP Now Primary Export Protocol

Liveness and Readiness Probes Enabled by Default

Controller/View Spans Silently Disabled

TIER 2

Tier 2 - Won't Start

4 cards

1. Actuator Endpoint @Nullable:
org.springframework.lang Replaced by JSpecify
2. AspectJ Weaving Required for @Observed
3. Controller & View Spans Disabled by Default
4. OTLP/HTTP Now Primary Export Protocol

Actuator Endpoint @Nullable: org.springframework.lang Replaced by JSpecify

Tier 2 - Won't Start

Observability & Ops

Impact: S

The migration guide says actuator endpoint parameters need org.jspecify.annotations.Nullable, but the deprecated org.springframework.lang.Nullable still works. Migrate as hygiene, not as a fix.

WHAT CHANGED

The Spring Boot 4.0 Migration Guide states that `org.springframework.lang.Nullable` no longer marks Actuator `@ReadOperation` / `@WriteOperation` parameters as optional, and that the supported annotation is now `org.jspecify.annotations.Nullable` (on the classpath transitively via Spring Framework 7.0). In practice the old annotation is still honoured on 4.0.0 and 4.0.7.

WHY

Spring Framework 7.0 migrated its codebase to JSpecify null-safety annotations and Spring Boot 4.0 followed. The `org.springframework.lang` annotations remain for source compatibility but are no longer the active standard.

SCOPE CHECK

Search for `@Nullable` in classes annotated with `@Endpoint`, `@WebEndpoint`, or `@JmxEndpoint`. The issue is specific to Actuator endpoint parameter binding.

WHAT YOU'LL SEE

```
// No error observed. On 4.0.0 and 4.0.7, an endpoint parameter
// annotated with @org.springframework.lang.Nullable is still treated
// as optional: GET without the parameter returns 200 (verified via MVC).
// The migration guide describes a break that does not reproduce.
```

THE FIX

Actuator endpoint parameter

BEFORE

```
import org.springframework.lang.Nullable;

@Endpoint(id = "my-endpoint")
public class MyEndpoint {
    @ReadOperation
    public String get(@Nullable String name) { ... }
}
```


AFTER

```
import org.jspecify.annotations.Nullable;

@Endpoint(id = "my-endpoint")
public class MyEndpoint {
    @ReadOperation
    public String get(@Nullable String name) { ... }
}
```

HOW TO FIX

Replace `org.springframework.lang.Nullable` with `org.jspecify.annotations.Nullable`.

Update the import on all Actuator endpoint parameters. Only the package changes; the annotation name and logic stay the same. JSpecify is already on the classpath via Spring Framework 7.0.

VERIFY

No observable break: an endpoint parameter annotated with the old `@Nullable` still returns 200 without the parameter on 4.0.0 and 4.0.7. Migrating to JSpecify is hygiene, not a fix

WATCH OUT

The old annotation works today only because it is meta-annotated with JSpecify's `@Nullable`. Do not rely on that: it is deprecated, and a later release could stop honouring it. Migrate now while it is a mechanical import swap.

FURTHER INFO

This is one piece of a wider null-safety overhaul, described in the [Spring Boot 4.0 blog post](#) on null-safe applications.

Migration guide: removed support for `org.springframework.lang.Nullable` Null-safe applications with Spring Boot 4

[Spring Boot 4.0 Migration Guide](#)

AspectJ Weaving Required for @Observed

Tier 2 - Won't Start

Observability & Ops

Impact: S

Spring Boot 4.0 requires AspectJ-based weaving for the `@Observed` annotation to produce spans and metrics. Without the aspect configured, `@Observed` does nothing.

WHAT CHANGED

In Spring Boot 3.x, `@Observed` worked via Spring AOP proxies and auto-configuration registered the `ObservedAspect` bean. In Spring Boot 4.0 that auto-configuration requires explicit AspectJ weaving support. Without the AspectJ weaving dependency and configuration, `@Observed` is ignored.

WHY

The change gives `@Observed` full AspectJ semantics: it now works on non-proxied beans, self-invocations, and private methods, the places where the proxy-based approach failed without warning.

SCOPE CHECK

Search for `@Observed` annotations across your codebase. Every `@Observed` method is broken until AspectJ weaving is configured. Also check whether dashboards or alerts depend on metrics or traces it produces.

WHAT YOU'LL SEE

```
# No startup error. Detected when @Observed methods
# produce no observations:

@Observed(name = "order.process")
public Order processOrder(OrderRequest request) { ... }

# Expected in metrics:
$ curl -s http://localhost:8080/actuator/metrics/order.process
{"name":"order.process","measurements":[{"statistic":"COUNT","value":42.0]}}

# Actual after upgrade:
$ curl -s http://localhost:8080/actuator/metrics/order.process
{"status":404,"error":"Not Found"}

# No metric registered, no spans created. The method runs
# normally; observability is gone.
```

THE FIX

Maven dependency — add AspectJ weaver

BEFORE

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing</artifactId>
</dependency>
```

AFTER

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing</artifactId>
</dependency>
<dependency>
  <groupId>org.aspectj</groupId>
  <artifactId>aspectjweaver</artifactId>
</dependency>
```

Configuration — register `ObservedAspect` if not auto-configured

BEFORE

```
// ObservedAspect was auto-configured in Boot 3.x
```

AFTER

```
@Configuration
public class ObservabilityConfig {
    @Bean
    public ObservedAspect observedAspect(ObservationRegistry registry) {
        return new ObservedAspect(registry);
    }
}
```

application.properties — enable AspectJ auto-proxy

BEFORE

```
# proxy-based AOP was sufficient
```

AFTER

```
spring.aop.auto=true
```

HOW TO FIX

Add AspectJ weaver dependency.

Add `org.aspectj:aspectjweaver` to your dependencies. Spring Boot's dependency management provides the version.

Register `ObservedAspect` bean explicitly.

If the auto-configuration does not pick up the aspect, register an `ObservedAspect` bean manually in a `@Configuration` class. Pass the `ObservationRegistry` to its constructor.

Verify AOP is enabled.

Ensure `spring.aop.auto=true` (the default) and that you have not disabled AspectJ auto-proxying. Check for `@EnableAspectJAutoProxy` if you have custom AOP configuration.

VERIFY

Custom `@Observed` aspects fire and metrics appear in `/actuator/metrics`

WATCH OUT

There is no error and no warning. The application starts, methods execute, and no observations (metrics or traces) are recorded. You discover it when monitoring dashboards show gaps.

Existing `@Timed` and `@Counted` annotations from Micrometer are separate from `@Observed` and may have their own configuration requirements. Check both.

If you use `@Observed` on `@Repository` or `@Service` beans that are JDK dynamic proxies, ensure AspectJ weaving mode is compatible. CGLIB proxies are generally safer.

Load-time weaving (`-javaagent:aspectjweaver.jar`) gives the broadest coverage but requires JVM argument changes. Compile-time weaving requires build plugin changes. Choose based on your deployment model.

FURTHER INFO

Part of the broader Micrometer Observation API refactoring in Spring Framework 7.0. See also: [controller-spans-disabled](#), [observability-dashboard-gaps](#).

[Spring-Break Demo](#) [Spring Boot 4.0 Migration Guide](#)

Controller & View Spans Disabled by Default

Tier 2 - Won't Start

Observability & Ops

Impact: S

Spring Boot 4.0 disables controller and view rendering spans by default in Micrometer tracing. Dashboards, alerts, and SLOs built on those spans go dark.

WHAT CHANGED

Spring Boot 4.0 changed the default for `management.observations.http.server.requests.name` and disabled the creation of child spans for Spring MVC controller method invocations and view rendering. Previously, Micrometer Tracing created spans for both the HTTP request and the controller method execution. Now only the HTTP server request span is created by default.

WHY

The controller and view spans added overhead and noise for applications that did not need sub-request granularity. Disabling them by default reduces trace volume and performance impact. Applications that need them can re-enable them explicitly.

SCOPE CHECK

Check your observability dashboards and alerts for references to controller-level span names. Search for span names like `controller.method.name`, `HandlerMethod`, or view rendering span names in your Grafana/Datadog/New Relic configuration. If you only use HTTP-level metrics (`http.server.requests`), you are not affected.

WHAT YOU'LL SEE

```
# No startup error. Detected when spans are missing
# in your observability platform:

$ curl -s http://localhost:16686/api/traces?service=my-app | jq '.data[0].spans[].operationName'
"HTTP GET /api/orders"
# Previously also showed:
# "OrderController#getOrders"
# "orders :: view rendering"
# These spans are now GONE.

# Grafana alert fires:
[FIRING] No controller-level spans for service my-app in last 15m
# Or SLO dashboard shows 0% data for controller latency percentiles.
```

THE FIX

application.properties — re-enable controller spans

BEFORE

```
# default: controller spans were enabled
```

AFTER

```
management.observations.http.server.requests.controller.enabled=true
management.observations.http.server.requests.view.enabled=true
```

application.yml equivalent

BEFORE

```
# default: controller spans were enabled
```

AFTER

```
management:
  observations:
    http:
      server:
        requests:
          controller:
            enabled: true
        view:
          enabled: true
```

HOW TO FIX

Re-enable controller spans.

Add `management.observations.http.server.requests.controller.enabled=true` to your `application.properties`. Do the same for `view.enabled` if you need view rendering spans. This restores the Spring Boot 3.x behaviour.

Update dashboards and alerts.

If you decide to accept the new default (no controller spans), update your Grafana dashboards, alert rules, and SLO definitions to use the HTTP server request span instead of controller-level spans. The HTTP span still contains the handler method in its attributes.

Use @Observed for targeted spans.

Instead of blanket controller spans, annotate specific controller methods with `@Observed` to get spans only where you need them. This is more efficient than re-enabling all controller spans globally.

VERIFY

Trace dashboard shows controller-level spans after re-enabling

WATCH OUT

The application gives no errors and no warnings. You discover the change when dashboards go blank or missing-data alerts fire.

Micrometer's `@Timed` annotation on controller methods still works for metrics. Only tracing spans are affected, not timer metrics.

If you use distributed tracing to debug latency between the HTTP layer and the controller, the missing span makes it harder to pinpoint where time is spent inside the server.

FURTHER INFO

Part of Spring Boot 4.0's Micrometer Tracing defaults overhaul. See also: [observability-dashboard-gaps](#), [aspectj-observed](#).

OTLP/HTTP Now Primary Export Protocol

Tier 2 - Won't Start

Observability & Ops

Impact: S

Spring Boot 4.0 changed the default OTLP transport from gRPC to HTTP/protobuf. Exports aimed at gRPC-only collectors get connection refused or vanish without an error.

WHAT CHANGED

Spring Boot 4.0 changed the default OTLP export transport from gRPC (port 4317) to HTTP/protobuf (port 4318). The default endpoint changed from

`http://localhost:4317` to `http://localhost:4318/v1/trace`s (and `/v1/metrics`, `/v1/logs`).

Applications that relied on the default gRPC transport now send data to the wrong port and protocol.

WHY

HTTP/protobuf is more universally supported, works through proxies and load balancers without special gRPC configuration, and does not require the gRPC dependency. The OpenTelemetry specification recommends HTTP/protobuf as the default transport.

SCOPE CHECK

Check your `application.properties` for `management.otlp` settings. If you have explicit endpoint configuration pointing to port 4317, the application will try HTTP on a gRPC port. If you have no explicit configuration, the default changed from gRPC/4317 to HTTP/4318.

WHAT YOU'LL SEE

```
WARN [otel.exporter] io.opentelemetry.exporter.otlp.http.OtlpHttpSpanExporter:
  Failed to export spans. Server responded with HTTP status 404.
  url=http://otel-collector:4318/v1/traces
---
# Or if collector only exposes gRPC port 4317:
WARN [otel.exporter] io.opentelemetry.exporter.otlp.http.OtlpHttpSpanExporter:
  Failed to export spans.
  java.net.ConnectException: Connection refused
  target=http://otel-collector:4318/v1/traces
---
# Metrics and traces stop arriving in your backend.
# Application continues running but observability data is lost.
```

THE FIX

`application.properties` — keep using gRPC

BEFORE

```
# default was gRPC on port 4317
```

AFTER

```
management.otlp.tracing.transport=grpc
management.otlp.tracing.endpoint=http://otel-collector:4317
management.otlp.metrics.export.transport=grpc
management.otlp.metrics.export.endpoint=http://otel-collector:4317
```

`application.properties` — accept new HTTP default

BEFORE

```
management.otlp.tracing.endpoint=http://otel-collector:4317
```

AFTER

```
management.otlp.tracing.endpoint=http://otel-collector:4318/v1/traces
```

docker-compose.yml — expose HTTP port on collector

BEFORE

```
otel-collector:
  image: otel/opentelemetry-collector:latest
  ports:
    - "4317:4317"    # gRPC
```

AFTER

```
otel-collector:
  image: otel/opentelemetry-collector:latest
  ports:
    - "4317:4317"    # gRPC
    - "4318:4318"    # HTTP/protobuf
```

HOW TO FIX

Accept HTTP/protobuf (recommended).

Update your OTLP endpoint to use port 4318 and the HTTP path format. Ensure your collector exposes the HTTP receiver. Most modern collectors (OpenTelemetry Collector, Grafana Agent, Datadog Agent) support both protocols.

Explicitly set gRPC transport.

If your collector only supports gRPC, add `management.otlp.tracing.transport=grpc` and set the endpoint to the gRPC port. This preserves the old behaviour.

Update collector configuration.

If using the OpenTelemetry Collector, ensure both the `otlp/grpc` and `otlp/http` receivers are enabled in your collector config. This lets you support both old and new applications during migration.

VERIFY

OTLP exporter sends traces to collector (check collector logs)

WATCH OUT

The application starts and runs normally. You only notice when traces and metrics stop appearing in your observability backend. Add health checks for telemetry export.

Some managed observability platforms (Datadog, New Relic) have different endpoints for HTTP vs gRPC OTLP ingestion. Check your provider's documentation for the correct HTTP endpoint URL.

If you use a service mesh or network policy that only allows port 4317, you need to open port 4318 as well.

The gRPC dependency (`io.grpc:grpc-netty-shaded`) may no longer be pulled in transitively. If you switch back to gRPC, you may need to add it explicitly.

FURTHER INFO

Affects spring-boot-starter-actuator consumers using OTLP tracing or metrics export.

Spring Boot 4.0 Migration Guide

TIER 3

Tier 3 - Behaves Differently

2 cards

1. Liveness and Readiness Probes Enabled by Default

2. Controller/View Spans Silently Disabled

Liveness and Readiness Probes Enabled by Default

Tier 3 - Behaves Differently

Observability & Ops

Impact: S

Boot 4.0 enables liveness and readiness probes everywhere, Kubernetes or not. The `/actuator/health` response changes shape. Downstream parsers and security rules may break.

WHAT CHANGED

The `management.endpoint.health.probes.enabled` property now defaults to `true` instead of being driven by Kubernetes detection. Both `/actuator/health/liveness` and `/actuator/health/readiness` are always exposed, and the main health response always lists them in the `groups` field.

WHY

Probes are useful outside Kubernetes: load balancers, service meshes, and health-check scripts benefit from explicit liveness and readiness semantics. Making them universally available removes the need for environment-specific configuration.

SCOPE CHECK

Search for code or tests that parse the `/actuator/health` response body and check for exact JSON structure. Also review Actuator security configurations for endpoint exposure rules. Check monitoring integrations (Datadog, Prometheus, PagerDuty health checks) that consume the health endpoint.

WHAT YOU'LL SEE

```
// Boot 3.5 (outside Kubernetes): /actuator/health response
{"status":"UP"}

// Boot 4.0: /actuator/health response includes probe groups
{
  "status": "UP",
  "groups": ["liveness", "readiness"]
}

// Consumer code that checked for exact JSON equality or did not
// expect the "groups" key now fails.
```

THE FIX

application.properties — disable probes if not wanted

BEFORE

```
# Boot 3.5: probes off by default outside Kubernetes
# management.endpoint.health.probes.enabled=true # required to enable
```

AFTER

```
# Boot 4.0: probes on by default; disable explicitly if not needed
management.endpoint.health.probes.enabled=false
```

HOW TO FIX

Disable probes if your deployment does not use them.

Set `management.endpoint.health.probes.enabled=false` in `application.properties` to restore Boot 3.5 behaviour for non-Kubernetes deployments. Alternatively, update downstream health consumers to expect the `groups` field.

Update security rules if probe endpoints were unintentionally exposed.

Review your actuator security configuration. If `/actuator/health/liveness` and `/actuator/health/readiness` were not expected to be publicly accessible, add them to your security exclusions or require authentication.

VERIFY

Health endpoint responds correctly and liveness/readiness groups are present or absent as expected after reviewing probe configuration

WATCH OUT

Application liveness state can be changed programmatically via `ApplicationAvailability`. If your code already does this, the probes are now always reachable: confirm the state transitions are correct before going to production.

FURTHER INFO

Boot 3.5 activated the probes only when it detected a Kubernetes environment (via environment variables) or when `management.endpoint.health.probes.enabled=true` was set explicitly. Boot 4.0 drops the detection, which can expose probe endpoints to networks that never expected them.

spring-break module: health-probes-default-on [Spring Boot 4.0 Migration Guide](#)

[Spring Boot Liveness and Readiness Probes](#)

Controller/View Spans Silently Disabled

Tier 3 - Behaves Differently

Observability & Ops

Impact: M

WHAT CHANGED

Spring Boot 4.0 disabled the automatic instrumentation for Spring MVC controller method spans and view rendering spans by default. The properties `management.observations.http.server.requests.enabled` and related `spring.mvc.observation` settings changed their defaults. The traces still record HTTP and JDBC spans, but the controller-level granularity is gone.

WHY

Controller spans added an extra span per request and duplicated information already in the HTTP server span. The new default cuts overhead and storage costs, with opt-in for teams that need the detail.

SCOPE CHECK

Check your observability dashboards, SLO definitions, and alerting rules. Any panel or alert that queries for controller-level span names (e.g. `controller.*`) or view rendering spans will show "No data" after the upgrade. Also check custom `SpanExporter` or `ObservationHandler` beans that filter by span name.

WHAT YOU'LL SEE

```
// Grafana / Jaeger trace — before (Spring Boot 3.5)
— HTTP GET /api/orders —————
  └─ controller: OrderController.getOrders [12ms]
    └─ view: orders/list [3ms]
      └─ jdbc: SELECT * FROM orders [8ms]

// Grafana / Jaeger trace — after (Spring Boot 4.0)
— HTTP GET /api/orders —————
  └─ jdbc: SELECT * FROM orders [8ms]
// controller and view spans MISSING

// Dashboard alert
"No data" for panel "Controller Response Time p99"
// SLO breach: "99th percentile latency unknown"
```

THE FIX

application.properties — re-enable controller spans

BEFORE

```
# (default in Boot 3.5: controller spans enabled)
```

AFTER

```
management.tracing.enabled=true
spring.mvc.observation.auto-configuration.enabled=true
```

Or enable via `ObservationRegistry` customiser

BEFORE

```
# (auto-instrumentation was on by default)
```

AFTER

```
@Bean
public ObservationRegistryCustomizer<ObservationRegistry> serverSpans() {
    return registry -> registry.observationConfig()
        .observationHandler(new DefaultMeterObservationHandler(meterRegistry));
}
```

Micrometer tracing config

BEFORE

```
# (no explicit config needed in Boot 3.5)
```

AFTER

```
management.observations.http.server.requests.enabled=true
management.observations.http.client.requests.enabled=true
```

HOW TO FIX

Re-enable controller observation spans.

Add `spring.mvc.observation.auto-configuration.enabled=true` and `management.observations.http.server.requests.enabled=true` to your `application.properties`. This restores the controller-level spans in your traces.

Update dashboards to use HTTP spans.

If the overhead matters, update your Grafana/Datadog dashboards to use the HTTP server span (`http.server.requests`) instead of the controller span. This span is still present and includes the handler information as attributes.

VERIFY

All Grafana/Jaeger panels show data: no 'No data' gaps

WATCH OUT

Logs and functionality look normal. The only symptom is missing data in your observability platform, often unnoticed until an incident needs the traces that are gone.

SLO alerts based on controller span latency percentiles now fire as "no data", which some alerting systems treat as "OK". Your SLO monitoring silently disappears.

FURTHER INFO

Part of Boot 4.0's Micrometer Tracing defaults overhaul. See also: [controller-spans-disabled](#), [aspectj-observed](#).

[Spring Boot 4.0 Migration Guide](#)



Not ready to migrate?

Spring Boot 3.5 stops receiving fixes when open-source support ends. You have three sane options. Pick one deliberately.

1. **Pin 3.5 and track CVEs.** Free, but the clock is running and every new CVE is your team's problem to triage.
2. **Migrate incrementally with OpenRewrite.** The Boot 4.0 recipe automates the renames in this book; the behavioural changes still need eyes.
3. **HeroDevs NES for Spring.** Security patches for 3.5 beyond EOL, buying your team time to migrate on your schedule, not the calendar's.

Spring Boot 4.0 Migration Guide • github.com/spring-projects/spring-boot/wiki
OpenRewrite Boot 4.0 recipe • docs.openrewrite.org/recipes/java/spring/boot4
HeroDevs NES for Spring • herodevs.com/support/spring-nes
The test suite behind this book • github.com/spoole167/spring-break